

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Analytical Problem Solving

Course Code:	CSA0306	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	Variganji Sumanth Babu	Reg. No.:	192524447
Section / Batch:	B. TECH – AI & DS	Date:	30 th July 2026

Code of Conduct

I Variganji Sumanth Babu / 192524447, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

V. Sumanth Babu

Signature of the Student:

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure	List Operations	1	20
Search Data Structure	Binary Search	2	20
Stack Data Structure	Stack Operations	3	20
Arithmetic Expression Evaluations	Stack Notations	4	20
Queue Data Structures	Priority Queue	5	20
GRAND TOTAL:			100 Marks

Annexure A – Analytical Problem Solving

1. Show the linked list configuration resulting from each series of list operations using the List ADT. Assume that lists L1 and L2 are empty at the beginning of each series. Show where the current position is in the list. (a) L1.append (10); L1.append (20); L1.append (15); (b) L2.append (10); L2.append (20); L2.append (15); L2.moveToStart (); L2.insert (39); L2.next (); L2.insert (12). (20 Marks)
2. Implement Binary searching Algorithm for searching an element and its position having the following 10 elements. Elements are 66, 111, 212, 323, 432, 543, 643, 649, 777, 888. Case 1: Key element = 111, Case 2: Key element=888, Case 3: Key element = 999. Analyse the concept in detail. (20 Marks)
3. Perform the following operations in LIFO data structure having an array size of 5. Push(45), Push(22), Push(11), Push(77), Pop(), Push(44), Push(33), Pop(), Pop(), Pop(). Print the peak element of the data structure and its index position. With a neat sketch explained in detail. (20 Marks)
4. Convert the expression $((a + b) + c * (d + e) + f) * (g + h)$ to the following expressions
 - a. Prefix expression
 - b. Postfix expression. (20 Marks)
5. A queue-based ticketing system serves customers in the order of arrival, but VIP customers must be prioritized occasionally. Analyze how modifying a simple queue into a priority-based queue affects fairness, waiting time, and computational complexity. Compare both systems using analytical reasoning. (20 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1 — Linked List ADT

1.1 Problem Understanding

Objective: We are given two singly linked lists, L1 and L2, manipulated through the standard List Abstract Data Type (ADT) operations `append()`, `insert()`, `moveToStart()`, and `next()`. Both lists start empty. We must trace the internal state of each list — the node sequence, the Head, the Tail, and the Current position (Curr) — after every single operation, and present the final configuration for each sequence.

Assumptions made:

- The List ADT follows the classical definition used in most Data Structures textbooks (Shaffer / Kruse style), where a list maintains a distinguished current position pointer in addition to Head and Tail.
- `append(x)` inserts `x` as the new last node of the list. It changes the Tail but does not alter the current position.
- `insert(x)` inserts `x` immediately before the node at the current position (i.e., at the current position), and the current position is then updated to refer to the newly inserted node.
- `moveToStart()` resets the current position to the first node of the list.
- `next()` advances the current position one node forward (no effect if already past the last node).

Edge cases considered: inserting/appending into an empty list (Head = Tail = new node), and `insert()` being called right after `moveToStart()` versus after a `next()`.

1.2 Data Structure Selection

Why a Linked List? A singly linked list is the natural choice for the List ADT here because the operations required — append, positional insert, and forward traversal (`next`) — do not require random access, only sequential node-to-node linkage via pointers.

Structure	Insert at Current Position	Remarks
Array-based list	$O(n)$ — requires shifting elements	Simple, but costly insert/delete
Singly Linked List	$O(1)$ once current position is located	Chosen: no shifting, only pointer relinking
Doubly Linked List	$O(1)$, plus $O(1)$ backward traversal	More memory (extra prev pointer); not needed since only <code>next()</code> is used

Since the question never requires moving backward, a singly linked list is sufficient and memory-efficient — a doubly linked list would be functionally correct too, but carries unnecessary overhead.

1.3 Algorithm Design — Operation Semantics

Operation	Effect on the List
<code>append(x)</code>	Create new node <code>x</code> ; link current Tail's next to it; update Tail = <code>x</code> . If list was empty, Head = Tail = <code>x</code> .
<code>insert(x)</code>	Create new node <code>x</code> ; link it before the node currently pointed to by Curr; Curr is updated to point to <code>x</code> .
<code>moveToStart()</code>	<code>Curr</code> ← Head.
<code>next()</code>	<code>Curr</code> ← <code>Curr</code> →next (advance by one node).

1.4 Step-by-Step Solution

Part (a): `L1.append(10)`; `L1.append(20)`; `L1.append(15)`;

Step 1 — `L1.append(10)`: List was empty, so the new node becomes both Head and Tail.

```
[ 10 ] → NULL
    ^Head, Tail
```

Step 2 — L1.append(20): A new node 20 is linked after the current Tail (10); Tail moves to 20.

```
[ 10 ] → [ 20 ] → NULL
    ^Head    ^Tail
```

Step 3 — L1.append(15): A new node 15 is linked after Tail (20); Tail moves to 15.

```
[ 10 ] → [ 20 ] → [ 15 ] → NULL
    ^Head                ^Tail
```

Final Configuration of L1 — List: 10 → 20 → 15 → NULL | Head = 10 | Tail = 15 | Curr = undefined (never set by moveToStart/insert in this sequence, since append does not affect it).

Part (b): L2.append(10); L2.append(20); L2.append(15); L2.moveToStart(); L2.insert(39); L2.next(); L2.insert(12);

Steps 1–3 are identical in nature to Part (a): three successive appends build the list 10 → 20 → 15 → NULL, with Head = 10 and Tail = 15.

Step 4 — L2.moveToStart(): Curr is reset to the Head.

```
[ 10 ] → [ 20 ] → [ 15 ] → NULL
    ^Head, Curr        ^Tail
```

Step 5 — L2.insert(39): A new node 39 is inserted immediately before the current node (10); Curr moves to the newly inserted node 39.

```
[ 39 ] → [ 10 ] → [ 20 ] → [ 15 ] → NULL
    ^Head, Curr  ^prev-Head        ^Tail
```

Step 6 — L2.next(): Curr advances one node forward, from 39 to 10.

```
[ 39 ] → [ 10 ] → [ 20 ] → [ 15 ] → NULL
    ^Head    ^Curr        ^Tail
```

Step 7 — L2.insert(12): A new node 12 is inserted immediately before Curr (10); Curr moves to 12.

```
[ 39 ] → [ 12 ] → [ 10 ] → [ 20 ] → [ 15 ] → NULL
    ^Head    ^Curr        ^Tail
```

Final Configuration of L2 — List: 39 → 12 → 10 → 20 → 15 → NULL | Head = 39 | Tail = 15 | Curr = 12

1.5 Complexity Analysis

Operation	Time Complexity	Reasoning
append(x)	O(1)	Tail pointer gives direct access to the last node
insert(x)	O(1)	Only pointer relinking at Curr; no shifting needed
moveToStart()	O(1)	Direct assignment Curr ← Head
next()	O(1)	Single pointer dereference

Space Complexity: $O(n)$ for n stored elements, plus $O(1)$ auxiliary pointers (Head, Tail, Curr).

Question 2 — Binary Search Algorithm

2.1 Introduction to Binary Search

Objective: Given the sorted array of 10 elements {66, 111, 212, 323, 432, 543, 643, 649, 777, 888} (indices 0 to 9), locate the position of three key values — 111, 888, and 999 — using the Binary Search algorithm, and analyse the search process in each case.

Binary Search is a divide-and-conquer searching technique applicable only to sorted arrays. At each step it compares the key with the middle element and eliminates half of the remaining search space, giving logarithmic-time performance — far superior to a linear scan for large datasets.

Assumptions: the array is 0-indexed and already sorted in ascending order (as given); mid is computed as $\lfloor (low + high) / 2 \rfloor$.

2.2 Data Structure Selection

Structure	Search Time	Remarks
Unsorted Array (Linear Search)	$O(n)$	No ordering exploited; simple but slow
Sorted Array (Binary Search)	$O(\log n)$	Chosen — ordering enables halving the search space each step
Binary Search Tree	$O(\log n)$ average	Needs extra pointer overhead; useful when frequent insert/delete also required

A sorted array is selected because the data is static (search-only) and array indexing gives $O(1)$ random access to the midpoint, which a linked structure cannot offer.

2.3 Algorithm Design — Pseudocode

```
Input : Sorted array A[0..n-1], key value K
Output: Index of K in A, or "Not Found"

1. Set low <- 0, high <- n-1
2. While low <= high:
    a. mid <- floor((low + high) / 2)
    b. If A[mid] = K: return mid          (found)
    c. Else if K < A[mid]: high <- mid - 1
    d. Else: low <- mid + 1
3. Return "Not Found"
```

Flow of Control

```
[ Start: low = 0, high = n-1 ]
    |
    +-----+
    | low <= high ? | --No--> [ Return Not Found ]
    +-----+
    | Yes
    [ mid = (low+high)/2 ]
    |
    +-----+
    | A[mid] == K ?      | --Yes--> [ Return mid (Found) ]
```

```

+-----+
|      | No
+-----+
| K < A[mid] ? | --Yes--> [ high = mid - 1 ]
+-----+
|      | No
[ low = mid + 1 ] -----> (repeat from low<=high?)

```

2.4 Step-by-Step Solution

Sorted Array:

Index	0	1	2	3	4	5	6	7	8	9
Value	66	111	212	323	432	543	643	649	777	888

Case 1: Key = 111 (Successful Search)

Iter.	low	high	mid	A[mid]	Comparison / Action
1	0	9	4	432	$111 < 432 \Rightarrow$ search left half; $high = mid - 1 = 3$
2	0	3	1	111	$111 = 111 \Rightarrow$ Key Found at index 1

Result: Key 111 found at index 1 in 2 comparisons.

Case 2: Key = 888 (Successful Search)

Iter.	low	high	mid	A[mid]	Comparison / Action
1	0	9	4	432	$888 > 432 \Rightarrow$ search right half; $low = mid + 1 = 5$
2	5	9	7	649	$888 > 649 \Rightarrow$ search right half; $low = mid + 1 = 8$
3	8	9	8	777	$888 > 777 \Rightarrow$ search right half; $low = mid + 1 = 9$
4	9	9	9	888	$888 = 888 \Rightarrow$ Key Found at index 9

Result: Key 888 found at index 9 in 4 comparisons.

Case 3: Key = 999 (Unsuccessful Search)

Iter.	low	high	mid	A[mid]	Comparison / Action
1	0	9	4	432	$999 > 432 \Rightarrow low = 5$
2	5	9	7	649	$999 > 649 \Rightarrow low = 8$
3	8	9	8	777	$999 > 777 \Rightarrow low = 9$
4	9	9	9	888	$999 > 888 \Rightarrow low = 10$
5	10	9	—	—	$low > high \Rightarrow$ loop terminates, Key Not Found

Result: Key 999 is not present in the array; search terminates after 4 comparisons when low exceeds high.

2.5 C Implementation

```
#include <stdio.h>

/* Returns index of key if found, else -1. Prints trace of each step. */
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    int iteration = 1;

    while (low <= high) {
        mid = (low + high) / 2;          /* compute middle index */
        printf("Iter %d: low=%d high=%d mid=%d arr[mid]=%d\n",
               iteration, low, high, mid, arr[mid]);

        if (arr[mid] == key)
            return mid;                  /* key found */
        else if (key < arr[mid])
            high = mid - 1;              /* search left half */
        else
            low = mid + 1;               /* search right half */

        iteration++;
    }
    return -1;                          /* key not found */
}

int main() {
    int arr[] = {66, 111, 212, 323, 432, 543, 643, 649, 777, 888};
    int n = 10;
    int keys[] = {111, 888, 999};

    for (int i = 0; i < 3; i++) {
        int result = binarySearch(arr, n, keys[i]);
        if (result != -1)
            printf("Key %d found at index %d\n\n", keys[i], result);
        else
            printf("Key %d not found in the array\n\n", keys[i]);
    }
    return 0;
}
```

Sample Output (abridged):

```
Key 111 found at index 1
Key 888 found at index 9
Key 999 not found in the array
```

2.6 Advantages, Limitations, Applications

Advantages	Limitations	Applications
Very fast, $O(\log n)$	Requires sorted data	Dictionary / word lookup
Few comparisons even for large n	Not efficient for frequent insert/delete on arrays	Searching in databases and indexes
Simple to implement iteratively or recursively	Poor performance on linked lists (no random access)	Debugging (bisection), version-control bisect

2.7 Complexity Analysis

Case	Time Complexity	Reasoning
Best Case	$O(1)$	Key is exactly at the first mid computed
Average Case	$O(\log n)$	Search space halves each iteration
Worst Case	$O(\log n)$	Key at an extreme end or absent; loop runs $\lceil \log_2 n \rceil$ times

Space Complexity: $O(1)$ for the iterative version (only a few index variables); $O(\log n)$ for a recursive version due to call-stack depth.

Question 3 — Stack (LIFO) Operations

3.1 Problem Understanding

Objective: Simulate a stack of fixed array size 5, performing the sequence: Push(45), Push(22), Push(11), Push(77), Pop(), Push(44), Push(33), Pop(), Pop(), Pop(). We must show the stack contents and TOP pointer after every operation, and finally report the peak (top) element and its index.

Assumptions: the stack is array-based with capacity 5 (valid indices 0–4); TOP = -1 denotes an empty stack; Push on a full stack causes Overflow; Pop on an empty stack causes Underflow.

3.2 Data Structure Selection

A Stack is the correct ADT because the problem explicitly follows the LIFO (Last-In-First-Out) discipline — the most recently pushed element must be the first one popped. An array-based implementation is chosen (over a linked list) because the capacity is fixed and known in advance (size 5), letting us detect Overflow/Underflow simply by checking TOP against the array bounds, with $O(1)$ access.

Implementation	Overflow Handling	Remarks
Array-based Stack	Natural — TOP == SIZE-1	Chosen: fixed size given in problem
Linked-list Stack	No overflow (bounded only by memory)	Unnecessary here; fixed size is specified

3.3 Algorithm Design — Push / Pop Pseudocode

Push(x): If TOP = SIZE-1, report Overflow. Else, TOP <- TOP + 1; Stack[TOP] <- x.
Pop() : If TOP = -1, report Underflow. Else, x <- Stack[TOP]; TOP <- TOP - 1; return x.

3.4 Step-by-Step Solution (Array size = 5, indices 0–4)

Step	Operation	TOP (index)	Stack Contents (bottom → top)	Remark
1	Push(45)	0	45	first element pushed
2	Push(22)	1	45, 22	
3	Push(11)	2	45, 22, 11	
4	Push(77)	3	45, 22, 11, 77	
5	Pop()	2	45, 22, 11	removes 77 (returned)
6	Push(44)	3	45, 22, 11, 44	
7	Push(33)	4	45, 22, 11, 44, 33	stack now FULL (size 5)
8	Pop()	3	45, 22, 11, 44	removes 33
9	Pop()	2	45, 22, 11	removes 44
10	Pop()	1	45, 22	removes 11

Diagrams of Key Stages:

After Step 4

TOP->	[77]
	[11]
	[22]
	[45]

After Step 7 (Full)

TOP->	[33]
	[44]
	[11]
	[22]
	[45]

Final (After Step 10)

TOP->	[22]
	[45]

3.5 Overflow and Underflow Conditions

- Overflow: occurs when Push is attempted while $TOP = SIZE - 1 = 4$. In this trace, the stack reaches exactly this state after Step 7 (elements 45, 22, 11, 44, 33) — any further Push without a Pop would cause Overflow.
- Underflow: occurs when Pop is attempted while $TOP = -1$ (empty stack). This does not occur in the given sequence since every Pop has a corresponding element available.

Final State — Final Stack (bottom → top): 45, 22 | Top Element = 22 | Top Index Position = 1

3.6 C Implementation

```
#include <stdio.h>
#define SIZE 5

int stack[SIZE];
int top = -1;          /* -1 indicates empty stack */

void push(int value) {
    if (top == SIZE - 1) {
        printf("Overflow! Cannot push %d\n", value);
        return;
    }
    stack[++top] = value;      /* increment top, then store */
    printf("Pushed %d, TOP now at index %d\n", value, top);
}

int pop() {
    if (top == -1) {
        printf("Underflow! Stack is empty\n");
        return -1;
    }
    return stack[top--];      /* return top value, then decrement */
}

int main() {
    push(45); push(22); push(11); push(77);
    printf("Popped %d\n", pop());
    push(44); push(33);
    printf("Popped %d\n", pop());
    printf("Popped %d\n", pop());
    printf("Popped %d\n", pop());

    if (top != -1)
        printf("Top element = %d at index %d\n", stack[top], top);
    return 0;
}
```

3.7 Complexity Analysis

Operation	Time Complexity	Space Complexity
Push	O(1)	O(1) auxiliary (array pre-allocated: O(n) total)
Pop	O(1)	O(1) auxiliary

Question 4 — Expression Conversion & Expression Tree

4.1 Problem Understanding

Objective: Convert the fully parenthesized infix expression

$$((a + b) + c * (d + e) + f) * (g + h)$$

into its Prefix and Postfix forms, construct the corresponding Expression Tree, and give its Preorder and Postorder traversals.

Operator precedence assumed: $*$ binds tighter than $+$; both are left-associative; explicit parentheses override precedence.

4.2 Data Structure Selection

Why a Stack and a Binary Tree? Infix-to-postfix/prefix conversion is a textbook application of the Operator Stack technique: operators are held on a stack until an operand of lower-or-equal precedence forces them to be popped into the output. Separately, the expression's structure is naturally a binary expression tree, where every internal node is an operator and every leaf is an operand — traversing this tree gives prefix (preorder), infix (inorder), and postfix (postorder) directly.

Approach	Mechanism	Remarks
Operator-stack (shunting-yard)	Stack holds operators & parentheses	Chosen for step-by-step conversion
Expression Tree	Tree holds operators (internal) & operands (leaves)	Chosen to visualize structure and derive traversals
Direct recursive parsing	Recursive descent grammar	More complex to hand-trace than a stack

4.3 Step-by-Step Postfix Conversion (Operator-Stack Method)

Algorithm — scan the infix expression left to right:

- Operand $\Rightarrow$ append directly to output.
- '(' $\Rightarrow$ push onto stack.
-)' $\Rightarrow$ pop and output until a matching '(' is popped (discarded).
- Operator $\Rightarrow$ pop and output all stack-top operators with greater or equal precedence, then push the current operator.
- At the end, pop all remaining operators to the output.

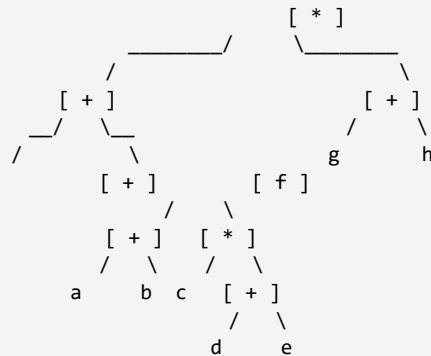
Token	Action	Operator Stack (top $\rightarrow$ right)	Output (Postfix so far)
(	push	(	
(	push	((	
a	output	((	a
+	push	((+	a
b	output	((+	a b
)	pop till (	) $\rightarrow$ a b +... see below	a b +
+	push (top is '(')	(+	a b +

Token	Action	Operator Stack (top→right)	Output (Postfix so far)
c	output	(+	a b + c
*	push (higher prec. than +)	(+ *	a b + c
(	push	(+ * (	a b + c
d	output	(+ * (	a b + c d
+	push	(+ * (+	a b + c d
e	output	(+ * (+	a b + c d e
)	pop till (	(+ *	a b + c d e +
+	pop *, pop + (equal prec.), push new +	(+	a b + c d e + * +
f	output	(+	a b + c d e + * + f
)	pop till ((empty)	(empty)	a b + c d e + * + f +
*	push (stack empty)	*	a b + c d e + * + f +
(	push	* (	a b + c d e + * + f +
g	output	* (	a b + c d e + * + f + g
+	push	* (+	a b + c d e + * + f + g
h	output	* (+	a b + c d e + * + f + g h
)	pop till (	*	a b + c d e + * + f + g h +
end	pop remaining (empty)	(empty)	a b + c d e + * + f + g h + *

*Postfix Expression: a b + c d e + * + f + g h + **

4.4 Expression Tree Construction

Grouping the expression by precedence: let $T1 = (a+b)$, $T2 = (d+e)$, $T3 = c * T2$, $T4 = T1 + T3$, $T5 = T4 + f$, $T6 = (g+h)$; the full expression is $T5 * T6$.



4.5 Prefix Conversion (via Preorder Traversal)

Preorder rule: visit Root, then Left subtree, then Right subtree. Applying this recursively to the tree above:

*Prefix Expression: * + + + a b * c + d e f + g h*

4.6 Postorder Traversal (Cross-check of Postfix)

Postorder rule: visit Left subtree, then Right subtree, then Root. Applying this to the same tree reproduces:

*a b + c d e + * + f + g h + **

This matches exactly the result obtained independently via the operator-stack method in Section 4.3, confirming correctness.

4.7 Complexity Analysis

Task	Time Complexity	Space Complexity
Infix $\rightarrow$ Postfix (stack method)	$O(n)$, n = number of tokens	$O(n)$ for operator stack
Tree construction	$O(n)$	$O(n)$ for tree nodes
Preorder / Postorder traversal	$O(n)$	$O(h)$ recursion stack, h = tree height

Question 5 — Queue vs. Priority Queue

5.1 Introduction

Objective: A queue-based ticketing system normally serves customers strictly in arrival order, but VIP customers occasionally need priority. We must analyse how replacing a simple (FIFO) queue with a priority queue affects fairness, waiting time, and computational complexity, and compare the two systems analytically.

5.2 Working Principle of a Simple Queue

A simple queue follows FIFO (First-In-First-Out): customers are served strictly in the order they arrive, via enqueue (arrival) at the rear and dequeue (service) from the front.

Front (served next) Rear (new arrivals join)
[C1] <- [C2] <- [C3] <- [C4]

5.3 Working Principle of a Priority Queue

A priority queue serves elements according to an assigned priority value rather than arrival order alone: a VIP customer (higher priority) can be moved ahead of ordinary customers already waiting. It is typically implemented with a heap for efficiency.

Served next
[VIP (P1)] <- [C1 (P2)] <- [C2 (P2)] <- [C3 (P2)]

5.4 Comparison Table

Aspect	Simple Queue	Priority Queue
Service order	Strict arrival order (FIFO)	By priority value; ties broken by arrival order
Fairness	Perfectly fair to all customers equally	Biased in favour of high-priority (VIP) customers
Waiting time (ordinary)	Predictable, proportional to arrival position	Can increase and become unpredictable if VIPs arrive frequently
Waiting time (VIP)	Same as any other customer	Minimized — served ahead of the general queue
Underlying structure	Circular array / linked list	Binary heap (most common), or sorted list
Enqueue complexity	$O(1)$	$O(\log n)$ (heap insert)
Dequeue complexity	$O(1)$	$O(\log n)$ (heap extract-min/max)
Starvation risk	None	Possible for low-priority customers under continuous VIP arrivals

5.5 Fairness Analysis

In a simple queue, fairness is absolute: position in line is the sole determinant of service order, so every customer's expected wait depends only on how many people arrived earlier. Introducing priority breaks this guarantee — an ordinary customer can be repeatedly overtaken by VIP arrivals, so the a-priori fairness of "first come, first served" is traded for a policy-driven fairness ("importance comes first"). Whether this is acceptable depends entirely on business policy, not on the data structure itself.

5.6 Waiting Time Analysis

Let λ be the arrival rate of ordinary customers and λ_v the arrival rate of VIP customers, with service rate μ .

- Simple Queue: average waiting time grows linearly with queue length ahead of a customer — roughly $W \approx n(\text{ahead}) / \mu$, independent of customer type.
- Priority Queue: VIP waiting time approaches $W(\text{VIP}) \approx n(\text{ahead, VIP}) / \mu$ (usually small, since few VIPs precede a VIP). Ordinary-customer waiting time increases because every newly arriving VIP can insert itself ahead: $W(\text{ordinary}) \approx [n(\text{ahead, ordinary}) + \text{VIP arrivals during wait}] / \mu$, which can grow unbounded if λ_v is high relative to μ (a classic queueing-theory starvation scenario).

5.7 Computational Complexity

Operation	Simple Queue	Priority Queue (Binary Heap)
Insert (arrival)	$O(1)$	$O(\log n)$
Remove (service)	$O(1)$	$O(\log n)$
Peek next-to-serve	$O(1)$	$O(1)$
Space	$O(n)$	$O(n)$

The priority queue trades a small increase in per-operation cost ($O(\log n)$ instead of $O(1)$) for the flexibility of serving by importance rather than only by arrival time — an acceptable overhead given n is typically small in a ticketing-counter context.

5.8 Advantages and Disadvantages

Priority Queue — Advantages	Priority Queue — Disadvantages
Critical/VIP requests handled promptly	Ordinary customers may face longer, unpredictable waits
Flexible: priority can encode business rules (VIP tier, urgency, etc.)	Higher implementation complexity (heap maintenance)
Still efficient: $O(\log n)$ operations scale well	Possible starvation of low-priority customers without ageing/fairness safeguards

5.9 Real-World Applications

- Simple Queue: bank teller lines, print job scheduling (basic), call-center hold lines.
- Priority Queue: hospital emergency triage, CPU process scheduling, Dijkstra's/Prim's shortest-path and MST algorithms, airline boarding priority, network packet QoS scheduling.

5.10 Conclusion

Converting a simple ticketing queue into a priority queue improves responsiveness for high-priority (VIP) customers at the direct cost of fairness and predictability for ordinary customers, while only mildly increasing computational overhead (logarithmic rather than constant time). The right choice depends on business goals: pure FIFO maximizes fairness, while a priority scheme maximizes service differentiation — a well-designed system often combines both, e.g., by ageing low-priority customers (gradually raising their priority the longer they wait) to bound worst-case waiting time and prevent starvation.